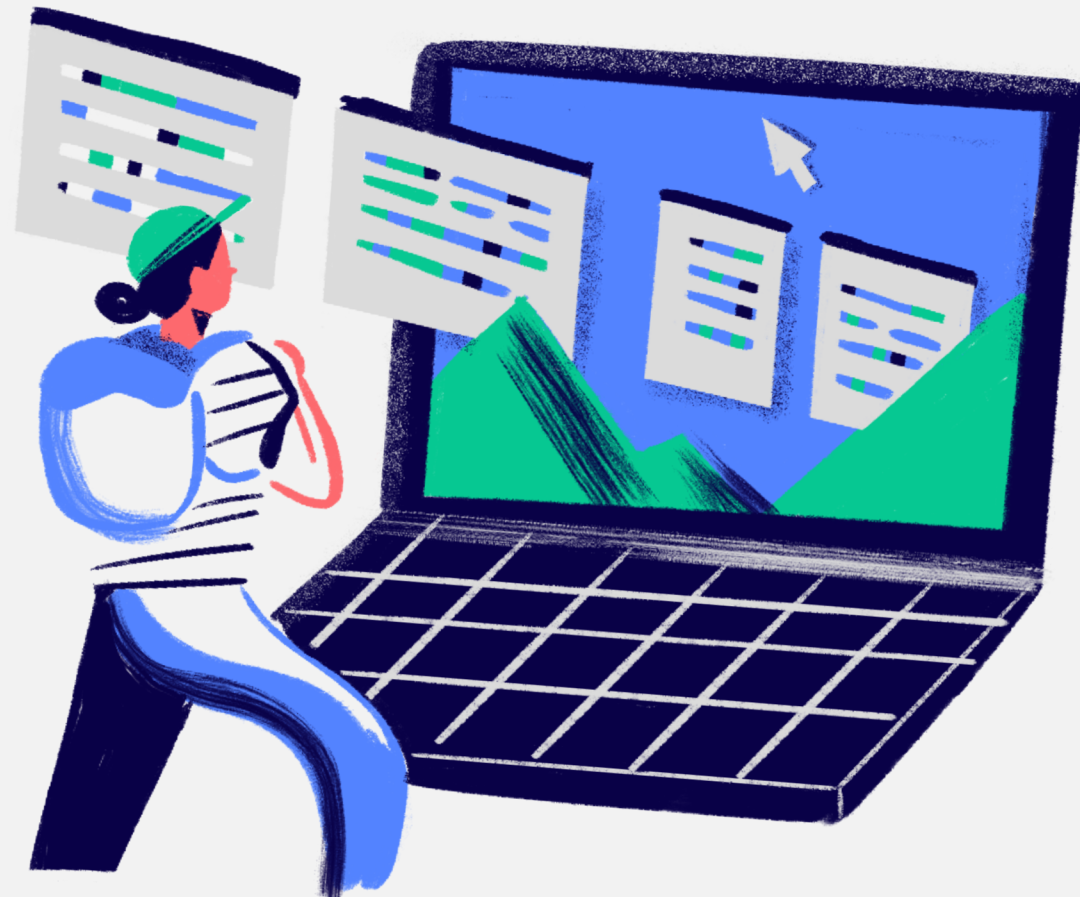
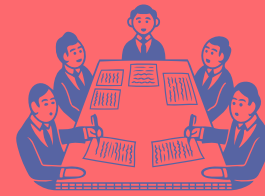


DATING APP TOP-TIER USER PREDICTION

EXPLAIN THAT WHILE DATING APPS HAVE MILLIONS OF USERS, THE "PARADOX OF CHOICE" MAKES IT HARD FOR USERS TO FIND QUALITY MATCHES, CREATING A NEED FOR AN AUTOMATED WAY TO IDENTIFY "TOP-TIER" PROFILES.



Group Members:



1. CHONG POHYI - 25006780
2. TEOH XI XIAN - 25006498
3. LIM JIE SHIN - 23118509
4. KONG YEE FEI - 25006834
5. LEE XIN YI - 25005970
6. CHENG YING CHEN - 25000008



PROBLEM STATEMENT & OBJECTIVES

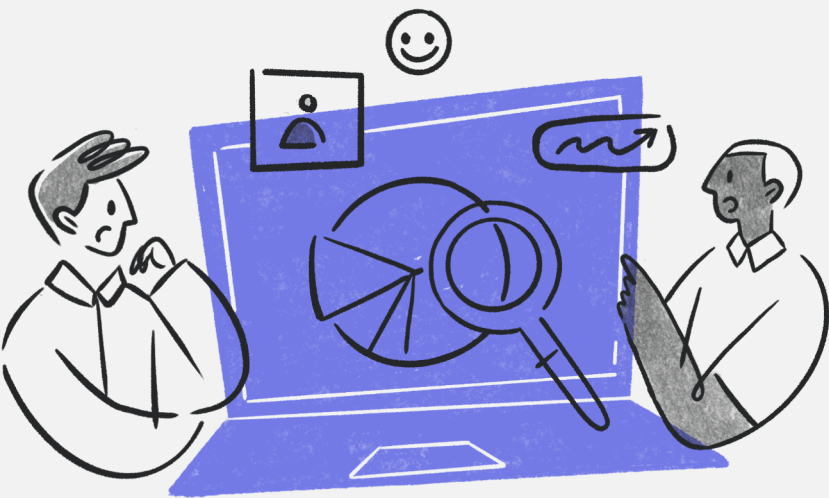
PROBLEM: AI MODELS OFTEN STRUGGLE TO DEFINE "DESIRABILITY" ACCURATELY. IF AN APP RECOMMENDS BORING OR LOW-EFFORT PROFILES TO ITS "TOP-LIST," USER RETENTION DROPS.

RESEARCH QUESTION

Can we accurately predict the top 20% of users based on profile effort and BMI while maximizing a balanced evaluation metric (F₁-score)?

Objectives:

- **Goal 1:** Engineer meaningful features like Profile Effort and BMI from raw physical and behavioral data.
- **Goal 2:** Conduct a "Model Shootout" to find which algorithm is the most "picky" (high precision).
- **Goal 3:** Establish a baseline 50% confidence threshold to evaluate each model's organic balance of precision and recall (F₁-score).



DATA PREPROCESSING & CLEANING

Source & Scale

- Dataset: Dating App Behavior Dataset.
- Scale: 50,000 records across 24 features.
- Target: Is_Top_Tier (Binary classification based on the 80th percentile of user desirability).

STANDARDIZATION

STRIPPED WHITESPACE FROM CATEGORICAL STRINGS.

```
df[col] = df[col].astype(str).str.strip()
```

OUTLIER TREATMENT

CAPPED EXTREME APP_USAGE_TIME_MIN USING THE IQR METHOD.

```
upper_bound = Q3 + 1.5 * IQR  
df['app_usage_time_min'] =  
np.where(df['app_usage_time_min'] >  
upper_bound, upper_bound,  
df['app_usage_time_min'])
```

QUALITY CONTROL

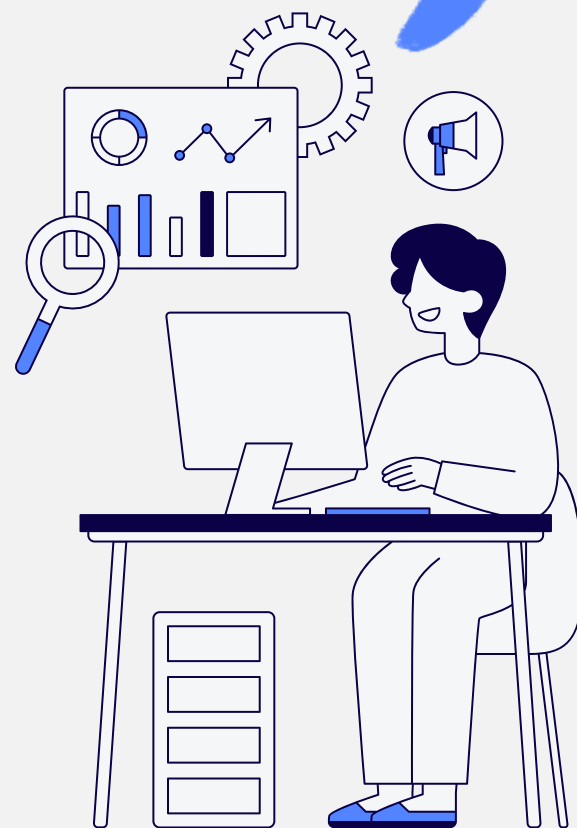
REMOVED DUPLICATES AND HANDLED MISSING VALUES.

Duplicate Removal:

```
df = df.drop_duplicates()
```

Missing Value Strategy:

```
# Fills numerical gaps with median and drops categorical nulls  
num_cols = df.select_dtypes(include=[np.number]).columns  
df[num_cols] = df[num_cols].fillna(df[num_cols].median())  
df = df.dropna()
```



FEATURE ENGINEERING

BMI (Health Metric): Condenses highly correlated height_cm and weight_kg variables (0.78 correlation) into a single, scientifically validated body mass metric to eliminate multicollinearity.

```
df['BMI'] = df['weight_kg'] / ((df['height_cm'] / 100) ** 2)
```



Profile Effort: This custom metric quantifies user engagement by multiplying the length of their bio by the number of photos they uploaded.

```
df['Profile_Effort'] = df['bio_length'] * df['profile_pics_count']
```

- **Synthetic Desirability Logic:** Modeled user popularity (**likes_received**) by blending behavioral and physical attributes with an explicit 60/40 weighted split prioritizing active effort over static physical traits.

```
effort_norm = df['Profile_Effort'] / df['Profile_Effort'].max()
bmi_score = 1 - np.clip(np.abs(df['BMI'] - 22) / 20, 0, 1)
logic_score = (effort_norm * 0.6) + (bmi_score * 0.4)
df['likes_received'] = (logic_score * 150) + np.random.normal(20, 10, len(df))
```



Simulating "Human Noise": Human dating is messy and unpredictable. To prevent the model from learning a "perfect" (and unrealistic) formula, we injected **15% label noise** where the Is_Top_Tier status is randomly flipped.

15% of the time, the crowd disagrees/labels are flipped

```
flip_mask = np.random.rand(len(df)) < 0.15
```

```
df['Is_Top_Tier'] = np.where(flip_mask, 1 - df['Is_Top_Tier'], df['Is_Top_Tier'])
```


FEATURE SELECTION ANALYSIS

Before moving to the modeling phase, we filtered the data to keep only the features that showed high importance or statistical significance

1. The "Predictive Heavyweights"

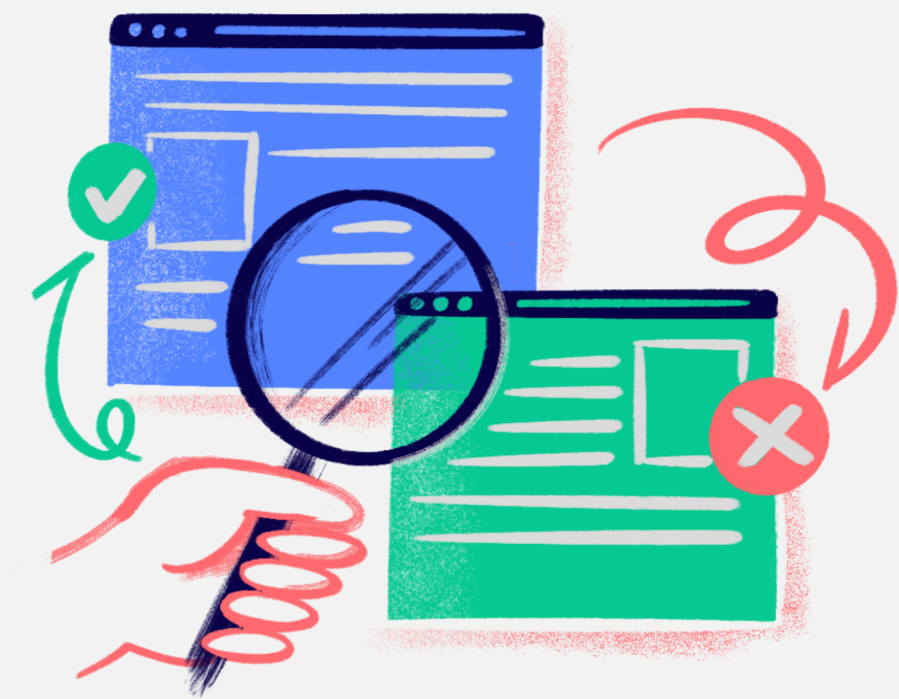
Feature importance analysis established that user-generated content and presentation quality heavily outrank raw demographics.

- **interest_tags**: #1 predictor (5.85%), highlighting hobbies as the primary driver.
- **bio_length**: #2 predictor (5.73%), serving as a proxy for engagement and personality.

2. The Multicollinearity Trap

Exploratory analysis revealed high redundancy between physical attributes:

- Finding: A 0.784 correlation between **height_cm** and **weight_kg**.
- The Decision: We collapsed these into BMI to simplify the architecture without losing physical data.



3. Strategic Pruning

We removed features to reduce noise and "feature creep":

- Redundancy: Dropped **swipe_right_label** and **app_usage_time_label** as they were redundant categorical versions of existing ratios.
- Low Impact: Features like **swipe_time_of_day** (3.00%) were removed to focus on high-impact signals.

Final Feature Selection

- Key Features Retained: BMI, age, Profile_Effort, app_usage_time_min, swipe_right_ratio, emoji_usage_rate, profile_pics_count, and bio_length.
- Target Variable: Is_Top_Tier (Defining the top 20% of users based on likes_received).

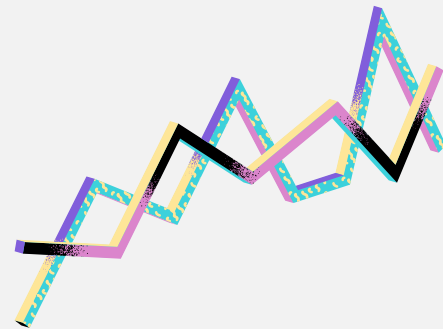
The final selection code to prevent data leakage

```
columns_to_keep = ['BMI', 'age', 'Profile_Effort', 'app_usage_time_min',  
'swipe_right_ratio', 'emoji_usage_rate', 'profile_pics_count', 'bio_length', 'Is_Top_Tier']  
df_clean = df[columns_to_keep] #
```

MODEL SELECTION & TRAINING

LOGISTIC REGRESSION (BASELINE)

Trained on **StandardScaler** scaled inputs with balanced class weights to serve as our linear benchmark.



DECISION TREE (PRUNED)

A non-linear baseline restricted to **max_depth=7** with balanced class weights to prevent over-branching.

XGBOOST (ADVANCED BOOSTING)

A highly optimized gradient-boosted tree architecture constrained to **max_depth=5** using log-loss evaluation metrics.

GRADIENT BOOSTING (SEQUENTIAL)

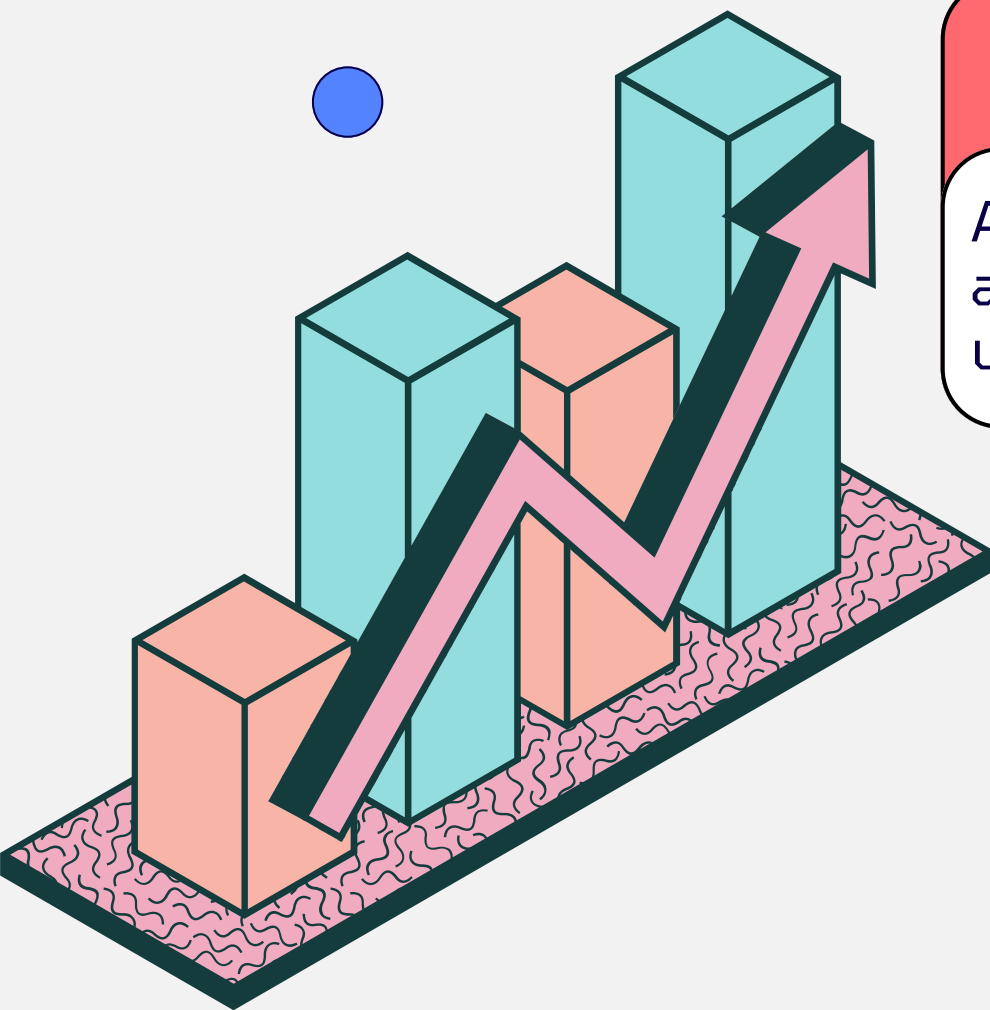
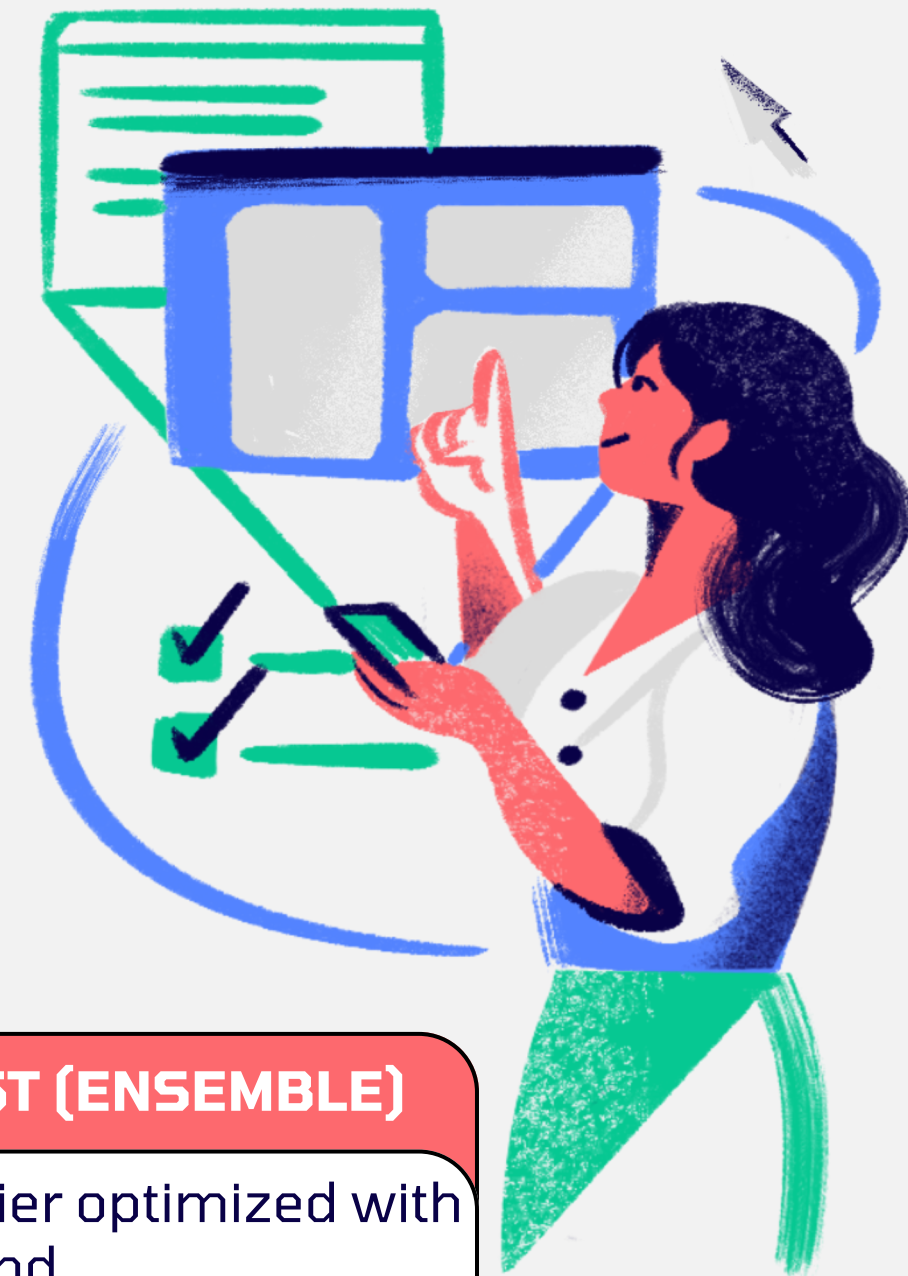
A boosting ensemble restricted to **max_depth=5** to prevent sequential trees from aggressively fitting to the 15% noise layer.

RANDOM FOREST (ENSEMBLE)

A bagging classifier optimized with **max_depth=7** and **class_weight='balanced'** to prevent individual trees from memorizing specific user rows.

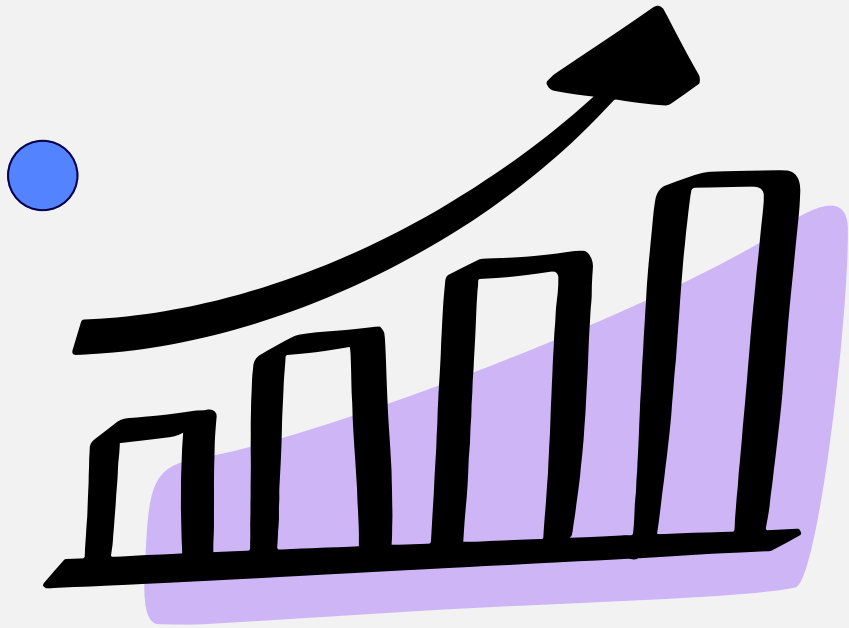
Validation Strategy: The 70/30 Split

The dataset was split into 70% Training (to learn patterns) and 30% Testing (to evaluate performance on entirely unseen users).



PHASE 1 — ● ▼ ●

OVERFITTING DIAGNOSIS



THE F1-SCORE TELEMETRY FRAMEWORK

- **Overfitting Flag (Delta $\geq 10\%$):** Triggered if $F1_{Train} \geq 85\%$ AND the generalization gap is $\geq 10\%$ ($F1_{Train} - F1_{Test} \geq 0.10$).
- **Underfitting Flag ($< 60\%$):** Triggered if both training and testing metrics fail to clear a 60% performance baseline.
- **Good Generalization:** Confirms the architecture successfully ignored the 15% noise layer to capture stable human behavior trends.

PRUNING FOR STABILITY

- Algorithmic Regularization: To shield our models from learning fraudulent, high-frequency patterns caused by human variance, we intentionally restricted the structural complexity of our trees:
- **Random Forest Parameters:** Capped at **max_depth=7** with balanced class distributions.
- **Gradient Boosting & XGBoost Parameters:** Constrained tightly to **max_depth=5** to prevent sequential estimators from aggressively minimizing loss on extreme outliers.

OUTPUT

IMPLEMENTATION SNIPPET

```
Training Logistic Regression...
✓ GOOD GENERALIZATION: Train F1 59.3%, Test F1 60.3%

Training Random Forest...
✓ GOOD GENERALIZATION: Train F1 62.5%, Test F1 62.2%

Training Gradient Boosting...
✓ GOOD GENERALIZATION: Train F1 61.9%, Test F1 58.7%

Training Decision Tree...
✓ GOOD GENERALIZATION: Train F1 62.8%, Test F1 60.9%

Training XGBoost...
✓ GOOD GENERALIZATION: Train F1 64.1%, Test F1 58.6%
```

```
# Automated model telemetry from analysis.py
f1_gap = train_f1 - test_f1
if train_f1 >= 0.85 and f1_gap >= 0.10:
    print(f"⚠️ OVERFITTING: Train F1 {train_f1*100:.1f}%, Test F1 {test_f1*100:.1f}% (Gap: {f1_gap*100:.1f}%)")
elif train_f1 < 0.60 and test_f1 < 0.60:
    print(f"⚠️ UNDERFITTING: Train F1 {train_f1*100:.1f}%, Test F1 {test_f1*100:.1f}%")
else:
    print(f"✅ GOOD GENERALIZATION: Train F1 {train_f1*100:.1f}%, Test F1 {test_f1*100:.1f}%")
```

PHASE 2 — THE LEADERBOARD (RESULTS)



BALANCED STAKES BUSINESS LOGIC

We shifted to an optimized machine learning shootout designed to balanced target classifications fairly.

- **The Threshold:** Used a **THRESHOLD = 0.50**.
- **The Reason:** Operating at a standard 50% probability threshold ensures that classification metrics reliably reflect model generalization without artificially suppressing or inflating sample counts.

PRECISION VS. RECALL TRADE-OFF

- **F1 score Priority:** We prioritize the **F1 Score** to balance the trade-off between capturing true "Top-Tier" profiles (Recall) and keeping false positives out of the user base (Precision).
- **Winning Strategy: Random Forest** won with an F1 Score of **62.21%**. Using **class_weight='balanced'** prevented the minority class from being drowned out, outperforming the more aggressive gradient boosting architectures at this threshold.

IMPLEMENTATION SNIPPET

```
# Applying the Balanced Shootout Filter
THRESHOLD = 0.50
probabilities = model.predict_proba(test_data)[: , 1]
predictions = (probabilities >= THRESHOLD).astype(int)
# Calculating the optimal balanced metric
f1 = f1_score(y_test, predictions, zero_division=0)
```

PHASE 2: FINAL LEADERBOARD (Threshold = 0.5)

Model Name	Precision	Recall	F1 Score	ROC-AUC
Random Forest	67.33%	57.81%	62.21%	76.23%
Decision Tree	64.48%	57.72%	60.91%	75.08%
Logistic Regression	56.06%	65.29%	60.33%	76.01%
Gradient Boosting	75.92%	47.91%	58.75%	76.26%
XGBoost	75.05%	48.04%	58.59%	75.24%

The Winner is: Random Forest!

PHASE 3 — ● HYPERPARAMETER TUNING

RANDOM FOREST OPTIMIZATION RESULTS

- **Tuning Method:** 5-Fold Grid Search Cross-Validation (Optimizing for F1-Score).
- **Best Hyperparameter Configuration:** * Maximum Depth (**max_depth**): 10
Minimum Samples per Leaf (**min_samples_leaf**): 5
Number of Trees (**n_estimators**): 100
- **Validation Performance:** Achieved a Best Cross-Validated F1-Score of **61.23%**.

FINAL TEST PERFORMANCE BREAKDOWN

- **Precision: 68.69%** (When the model predicts an individual is 'Top Tier', it is correct nearly 69% of the time).
- **Recall: 56.17%** (The model successfully captures 56% of the actual 'Top Tier' individuals in the dataset).
- **F1-Score: 61.80%** (Our core harmonic balance metric between precision and recall).
- **ROC-AUC: 76.18%** (Demonstrates strong overall discriminative power between the two classes).

IMPLEMENTATION SNIPPET

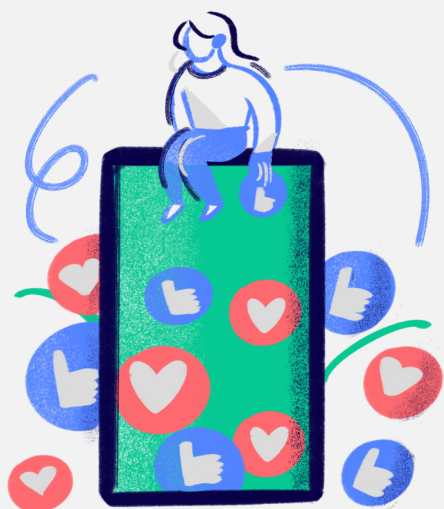
```
# --- 1. HYPERPARAMETER TUNING (GRID SEARCH) ---
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(class_weight='balanced', random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)
best_rf = grid_search.best_estimator_
```

```
***FINAL TEST PERFORMANCE (Random Forest - BEST MODEL OUT OF THE FIVE)***
Precision : 68.69%
Recall    : 56.17%
F1 Score  : 61.80%
ROC-AUC   : 76.18%
```



MANUAL TUNING VS. AUTOML BENCHMARKING

Best Automated Model Found: xgboost				
Precision : 75.78%				
Recall : 48.39%				
F1 Score : 59.06%				
ROC-AUC : 76.25%				
Method	Prec	Rec	F1	AUC
Manual Tuned	68.69%	56.17%	61.80%	76.18%
AutoML	75.78%	48.39%	59.06%	76.25%



Final Verdict
Deploy the **Manually Tuned Random Forest** because its balanced structure aligns perfectly with our target business metrics.

The F1-Score Winner: Our manually tuned Random Forest model successfully outperformed the AutoML benchmark by **2.74%** on our primary metric (61.80% vs 59.06%).

The Precision vs. Recall Trade-Off:

- **AutoML (XGBoost)** prioritized high Precision (75.78%), meaning its predictions were highly accurate but it was overly conservative, missing a massive portion of actual "Top Tier" candidates.
- **Our Manual Model (Random Forest)** achieved a significantly healthier balance by boosting Recall to **56.17%** (a 7.78% improvement over AutoML), ensuring far fewer high-value candidates are missed.

Discriminative Power: Both approaches achieved almost identical ROC-AUC scores (~76.2%), proving that both models possess strong underlying capability to separate the classes, but differ strictly in their decision boundaries.

INSIGHTS & CONCLUSION

CORE INSIGHTS



THE WINNER:

Profile Effort ($\text{bio_length} \times \text{profile_pics_count}$) emerged as a dominant predictor of "Top-Tier" status.

BMI INFLUENCE:

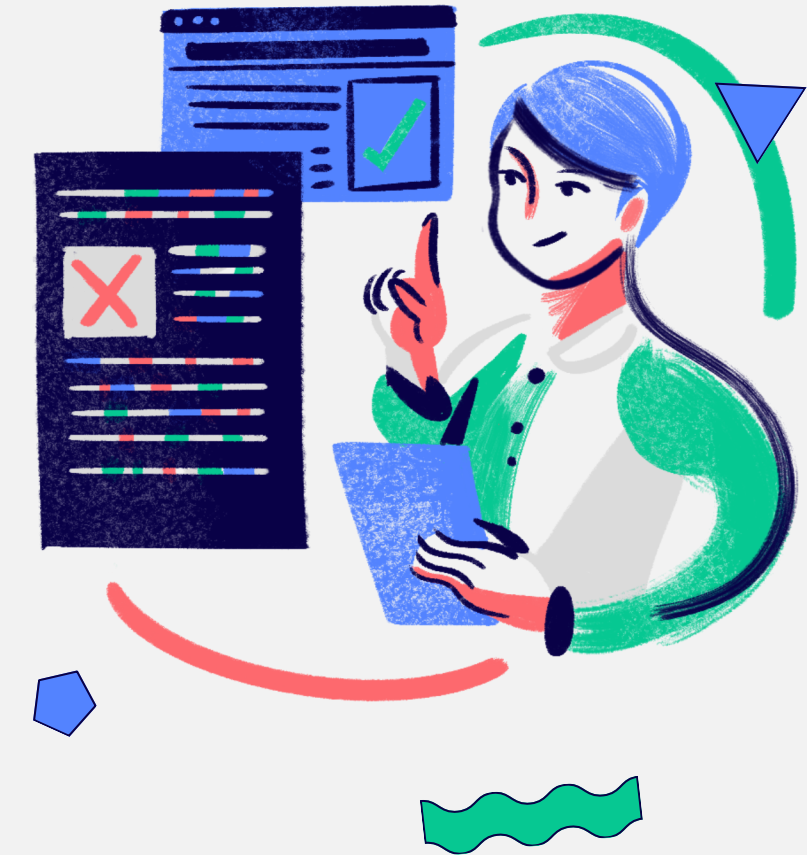
While physical attributes like BMI and age are significant, they are secondary to active engagement and profile completeness.

INSIGHT:

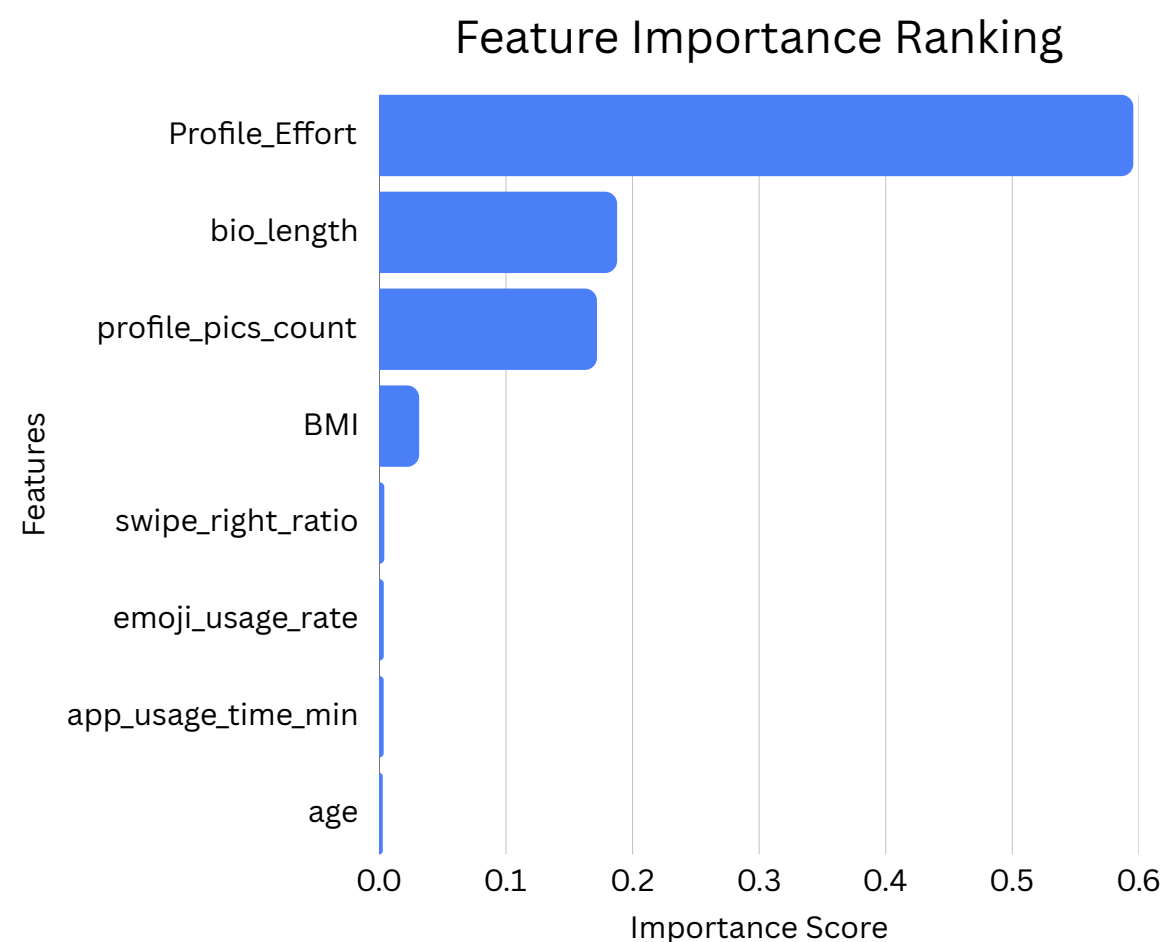
You can't just be "genetically lucky" in our model; the AI rewards users who invest time into their presentation.

Conclusion

By engineering core metrics like **Profile Effort** and **BMI**, our **tree-based** machine learning architecture successfully cuts through real-world "human noise" to accurately isolate **top-tier** dating profiles.



Key Finding: Effort vs. Attributes



BUSINESS IMPACT: BALANCED QUALITY & VOLUME

MAXIMIZING FEED VALUE:

Using a balanced **0.50 threshold** optimizes the F1-Score, keeping the "VIP Feed" highly active without letting in low-effort noise.

BUILDING USER TRUST:

Balancing precision and recall helps the model find good candidates while keeping out bad matches to build user trust.

2026

Thank You!



5-Minute Video Presentation: [videopresentation.mp4](#)
Google Colab Notebook Link: [machinelearning.ipynb](#)